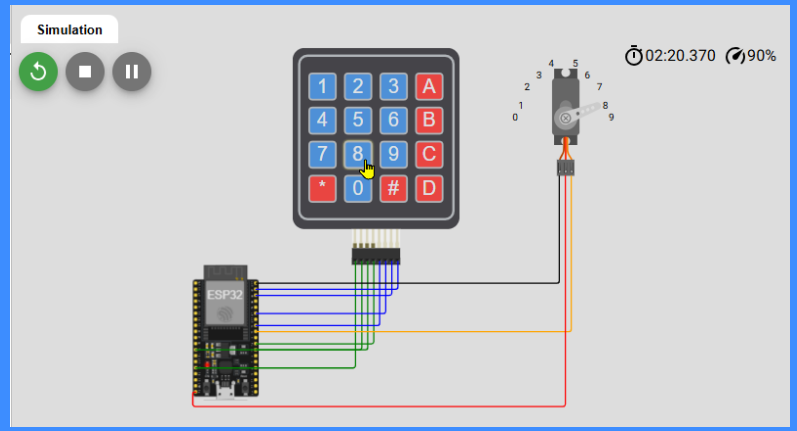
ESP32 / WOKWI PROJECT

Press a key.
Move the servo.

A development story from Arduino reference video to a working ESP32 circuit

KEYPAD → CONTROL → MOTION



ESP32 Key-Activated Servo

How do we turn one keypress into visible motion?

The project starts with a short video reference and ends with a documented ESP32/Wokwi build.

INPUT

4×4 keypad

A compact matrix of 16 keys

LOGIC

ESP32

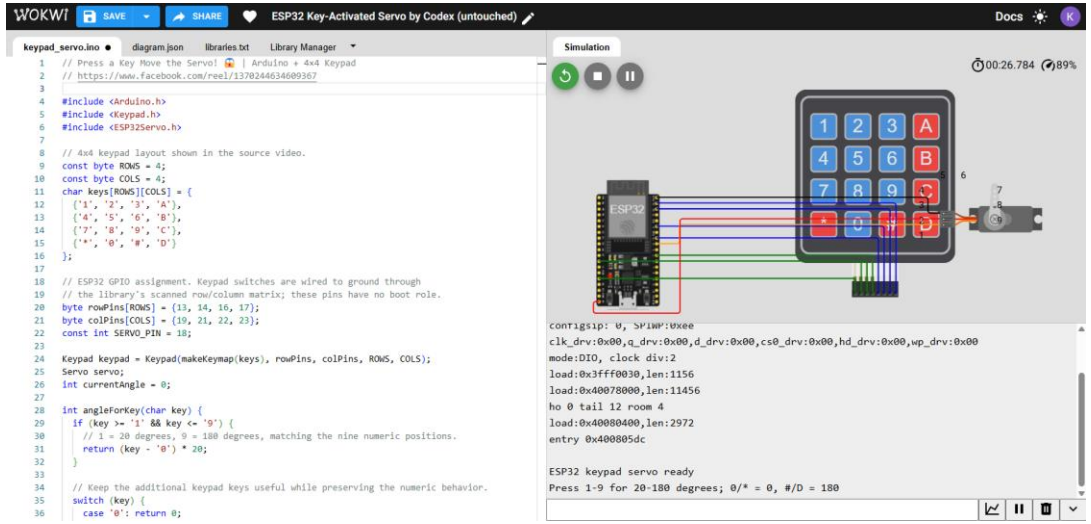
Scan → map → report

OUTPUT

Servo angle

A physical position per command

The source video establishes the interaction—not the final hardware.



WHAT THE VIDEO GIVES US

- 4×4 keypad as the input surface
- Servo position changes after a keypress
- Arduino Uno wiring and code visible in the reel
- No accompanying written source code

Design consequence: behavior had to be extracted from video evidence before porting.

The ESP32 port preserves the interaction while changing the platform assumptions.

REFERENCE PLATFORM

Arduino Uno

Video wiring and code cues

source behavior

PORT

FINAL PLATFORM

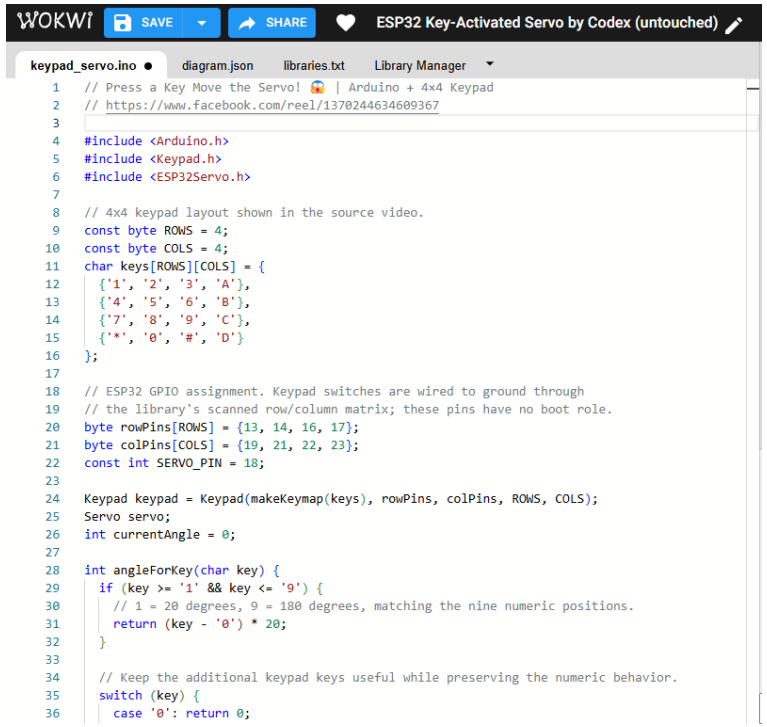
ESP32 DevKit C V4

ESP32Servo + Keypad libraries

Wokwi-ready

Porting rule: keep the user-visible behavior; replace board-specific pins and APIs.

The unmodified Wokwi circuit makes the port concrete—but exposes wiring and presentation gaps.

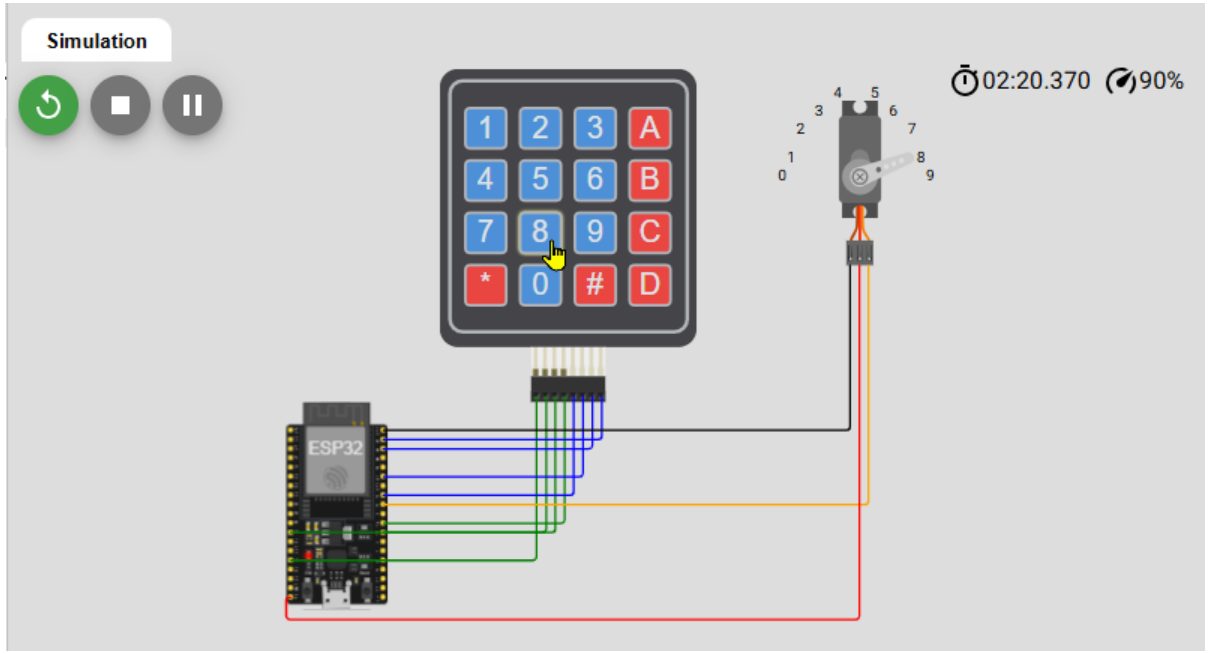


UNMODIFIED BUILD

- ESP32, keypad, and servo are present
- Initial routing was difficult to read
- Keypad connections were not visibly rendered
- Serial Monitor behavior was unclear

The next iteration is a touch-up pass, not a change in project intent.

Reference circuits resolved the connection format; layout changes made the result readable.



WHAT CHANGED

- Numeric ESP32 endpoints match references
- Keypad wires are visible and color-separated
- Servo moved away from keypad for clarity
- Angle labels 0–9 added around the servo
- Serial-monitor links retained for runtime output

The final circuit is a simple three-stage loop: scan, translate, actuate.

01

Scan keypad

keypad.getKey()

02

Map character

angleForKey()

03

Move servo

servo.write(angle)

One press → one command

The BOM stays intentionally small: one controller, one input surface, one actuator.

COMPONENT	QTY	ROLE
ESP32 DevKit C V4	**1**	Controller
4×4 membrane keypad	**1**	User input
Micro servo	**1**	Angle output
Wires	**As required**	Signal + power

The “touch-up” work changes readability and confidence—not the BOM.

The final map avoids the reference board’s Arduino pin assumptions while keeping the matrix model intact.

FINAL ESP32 MAP

R1–R4	**13, 14, 16, 17**
C1–C4	**19, 21, 22, 23**
Servo PWM	**18**
Servo V+ / GND	**5V / GND.2**

REFERENCE PATTERN USED

- Keypad reference: matrix pins map directly to R1–R4 and C1–C4
- Servo reference: GPIO 18 + 5V + common ground
- Serial reference: TX/RX connected to the monitor endpoints

The keypad becomes a compact angle selector with predictable feedback.

NUMERIC KEYS

1	20°	2	40°	3	60°
4	80°	5	100°	6	120°
7	140°	8	160°	9	180°

OTHER KEYS

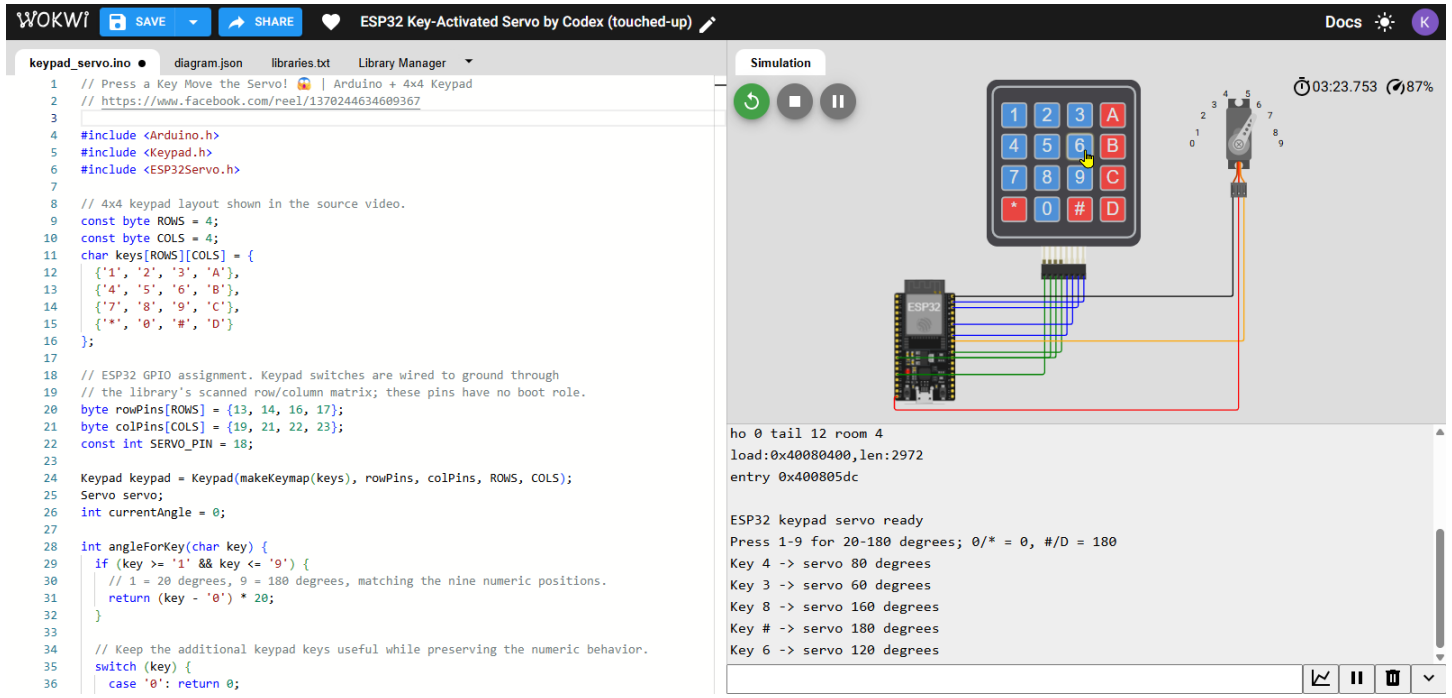
0 / *	0°
A	45°
B	90°
C	135°
D / #	180°

Serial output reports the accepted key and target angle.

Each issue produced a concrete rule for the final build.

Invisible keypad wires	Use reference-compatible numeric GPIO endpoints	Readable routing
Serial Monitor absent	Use TX/RX monitor links; open after simulation starts	Observable runtime
Held key repeats	Wait for release + debounce delay	One press = one command
Board/API mismatch	Use ESP32 DevKit C V4 + ESP32Servo	Portable port

The touched-up circuit makes the behavior inspectable in one view.



VISIBLE PROOF

- Keypad wires are visible
- Servo labels 0–9 are visible
- Serial output confirms angles
- Code and circuit share the same map

Evidence is presented without claiming that Codex ran Wokwi live.

The project gained portability and clarity without changing the core interaction.

CHANGED

- Arduino Uno reference → ESP32 DevKit C V4
- Board-specific pins → ESP32 GPIO map
- Initial layout → visible, aesthetic routing
- Unclear monitor → explicit serial links

STAYED STABLE

- 4×4 keypad remains the user input
- A key still maps to a servo position
- Servo remains the visible output
- The design stays compact and teachable

Follow the evidence from source reference to final circuit.

SOURCE

Facebook Reel — Press a Key, Move the Servo!

WOKWI REFERENCES

Keypad input reference

ESP32 servo reference

CURRENT PROJECT BUILDS

ESP32 Key-Activated Servo — unmodified

ESP32 Key-Activated Servo — touched up

TAKEAWAY

A small interaction can carry a full engineering story.